

Course Dashboard & Content Editor

Implementation guide — a data-driven curriculum authored through the admin UI

Prepared 7 August 2026 · Derived from the design session on the Figma prototype and the `.dc.html` page set

1. Purpose

Today every screen hard-codes its own content. This document specifies how to replace that with a **single curriculum object** owned by the backend, rendered by the learner app, and edited visually by an administrator through a two-page admin UI: a **Course Dashboard** (statistics and structure) and a **Content Editor** (per-item editing with live preview).

The target outcome is that adding a lesson, rewording a question, changing an answer, reordering the roadmap, or introducing a whole new kind of exercise are all *content operations* performed in the UI — not code changes and not deployments.

2. What is wrong today

Four concrete problems in the current prototype set motivate the whole design:

Problem	Where it shows up
Duplicated content	The same <code>STAGES</code> array is copy-pasted into <code>Course Roadmap.dc.html</code> and <code>Stage Lessons.dc.html</code> . Editing one silently desynchronises the other.
Incompatible question shapes	<code>Exercise.dc.html</code> uses <code>type: 'multiple-choice' 'gap-fill'</code> with <code>correct / correctFeedback</code> ; <code>Placement Test.dc.html</code> uses <code>type: 'choice' 'translate'</code> with different field names. Neither can be authored generically.
Index-based identity	Navigation uses <code>?stage=2</code> and lookups use <code>STAGES[idx]</code> . The moment an admin reorders stages, every stored progress record points at the wrong content.
Answer keys in the client	<code>correct:</code> ships inside the page source and is readable in developer tools.

3. Principles

- 1 Identity is a slug; order is a field.** Every stage, lesson and exercise carries a stable `id` that is never reused and never renumbered, plus a separate `order`. Reordering changes `order` only, so progress records stay valid. This single split is what makes the roadmap editable at all.
- 2 One envelope, a typed payload.** Every exercise shares the same outer shape and differs only in `payload`,

discriminated by `type` .

- 3 **The engine never knows the types.** Adding an exercise kind means adding registry entries, never editing the runner, the roadmap, or the schema.
- 4 **Content is edited; analytics are observed.** They are separate objects with separate lifecycles and separate endpoints, and must never be merged.
- 5 **Learners read a published snapshot.** Admins edit a draft. Publishing validates and versions. Attempts record the version they were answered against.

4. Data model

4.1 The curriculum tree

```
type Loc = { en: string; si: string }; // every learner-visible string is bilingual

type Curriculum = { version: number; publishedAt: string; stages: Stage[] };

type Stage = {
  id: string; // "course" in the admin UI
  order: number; // "tenses" – stable slug, never reused
  title: Loc; // display position; safe to change
  theme: { from: string; to: string }; // planet gradient on the roadmap
  unlock: UnlockRule;
  lessons: Lesson[];
};

type UnlockRule =
  | { kind: 'always' }
  | { kind: 'afterStage'; stageId: string }
  | { kind: 'minXp'; xp: number };

type Lesson = {
  id: string; order: number; title: Loc;
  kind: 'teach' | 'practice' | 'checkpoint' | 'writing';
  explanation: Loc;
  examples?: { sinhala: Loc; english: Loc; tag: Loc }[];
  commonMistake?: Loc;
  exercises: Exercise[]; // or exerciseIds[] once an exercise bank exists
};
```

4.2 The exercise envelope

```
type Exercise = {
  id: string;
  type: 'mcq' | 'gap_fill' | 'drag_order' | 'match' | 'free_text';
  prompt: Loc;
  feedback: { correct: Loc; incorrect: Loc };
  hint?: Loc;
  payload: Payload; // discriminated by `type`
};
```

type	UI label	Payload	Answer key field
mcq	Multiple choice	{ options: Loc[], correctIndex, shuffle }	correctIndex
gap_fill	Fill in the blank	{ template: Loc, accept: string[], choices: string[] }	accept
drag_order	Drag to order	{ tokens: Loc[] } — stored in correct order	token order
match	Match pairs	{ pairs: { left, right }[] }	pair alignment
free_text	Free text	{ accept: string[], normalize: { lowercase, stripPunctuation } }	accept

DESIGN NOTE

gap_fill stores the sentence with a literal `___` marker rather than a parsed token list. Admins type the sentence naturally; the renderer splits on the marker. The existing word-order question (*"I rice eat / I eat rice"*) is a better drag_order than an mcq — migrating it is a good first test of the model.

4.3 Analytics — a separate object

```
Analytics = {
  activeLearners, weeklyActive, avgSessionMin,
  byStage: { [stageId]: { learners, completionPct, avgAccuracyPct } }
};
```

Derived from the attempts and progress tables. It is read-only to the dashboard and served by its own endpoint, because content is authored while analytics are measured.

5. The type registry — the core extension point

One `type` string keys three parallel registries, one per concern:

```
backend/graders/  mcq.ts  gap_fill.ts  drag_order.ts  match.ts  free_text.ts
frontend/players/ Mcq     GapFill     DragOrder     Match     FreeText
admin/editors/   McqForm GapFillForm DragOrderForm MatchForm FreeTextForm

// frontend runner — no knowledge of any specific type
const PLAYERS = { mcq: Mcq, gap_fill: GapFill, drag_order: DragOrder,
                  match: Match, free_text: FreeText };
const Player = PLAYERS[exercise.type];
```

The exercise runner only ever does: show prompt → collect response → submit → show feedback → advance. A fourth registry, `DEFAULT_PAYLOAD[type]()`, supplies the blank payload used when an admin switches an exercise from one type to another.

ACCEPTANCE TEST

Adding a sixth exercise kind must touch **only** the four registry entries and the three new files. If it forces a change to the runner, the roadmap, or the schema, the abstraction has leaked.

6. Answer-key security

Answer keys must be stripped from the public read path and grading must happen on the server:

```
GET  /api/curriculum?lang=en  →  exercises with every answer-key field REMOVED
POST /api/attempts            →  { exerciseId, response }
                                   ←  { correct, feedback, xpAwarded }
```

The same grader module runs for practice and for assessment. Instant client-side feedback may be enabled per lesson via a `gradeClientSide` flag for offline feel, but the **placement test and checkpoints must always be graded server-side** — those results determine a learner's starting stage.

7. Persistence

Relational where integrity matters, JSON where the shape varies by type:

```
stages(id PK, order, title_en, title_si, theme JSONB, unlock JSONB)
lessons(id PK, stage_id FK, order, kind, title_en, title_si, body JSONB)
exercises(id PK, type, prompt JSONB, feedback JSONB, payload JSONB)
lesson_exercises(lesson_id FK, exercise_id FK, order)

attempts(user_id, exercise_id, content_version, response JSONB, correct, at)
progress(user_id, stage_id, lesson_id, status, pct)
```

PostgreSQL on RDS fits the planned AWS deployment. Validate `payload` against a JSON Schema (or Zod) definition shared by the editor, the API and the grader — one definition, three consumers.

Draft and publish

The admin edits a **draft**. `POST /api/admin/publish` validates the whole tree, snapshots it and increments `version`. Learners only ever read the published snapshot. Every attempt stores `content_version`, so editing a question never rewrites history or corrupts past progress percentages.

API surface

Endpoint	Purpose
<code>GET /api/curriculum</code>	Published tree, answer keys stripped, ETag-cacheable
<code>POST /api/attempts</code>	Grade one response, return correctness and feedback
<code>GET /api/admin/curriculum</code>	Full draft including answer keys (authenticated)
<code>PUT /api/admin/curriculum</code>	Save the draft from the editor
<code>POST /api/admin/publish</code>	Validate, snapshot, bump version
<code>GET /api/admin/analytics</code>	Learner metrics for the dashboard

8. Course Dashboard — specification

The dashboard answers “what is in the course, and how is it doing?”, then hands off to the editor.

8.1 Statistics band

Four stat tiles as hero numbers with a context line beneath: **Courses**, **Lessons**, **Exercises** (subtitled with how many of the five types are in use), and **Active learners**. The first three are computed from the curriculum; the fourth comes from analytics.

8.2 Exercise mix

A horizontal bar chart of exercise count per type, one row per type, always showing all five rows so an unused type is visible as a zero rather than absent. Bars are a **single hue** with the count direct-labelled, and are scaled against the largest count. See §11 for why one hue and not five.

8.3 Content checks

Validation run against the live draft, each row an icon plus a coloured headline plus an explanation — never colour alone. Minimum set:

- Prompts missing a Sinhala translation (learners on the SI toggle silently fall back to English)
- Exercises missing correct or incorrect feedback
- Lessons containing no exercises (skipped in the learner roadmap)
- Courses containing no lessons (can never be unlocked)

These are the same rules `POST /api/admin/publish` should enforce as hard gates.

8.4 Course list

One card per course showing order badge, bilingual title, lesson count, exercise count and a learner completion meter, with an **Edit course** action. Expanding a card reveals its lessons — order, title, explanation excerpt, the exercise types it contains as chips, exercise count, and an **Edit** action. + **Add course** and + **Add lesson** create a stub with a generated unique id and persist immediately.

9. Content Editor — specification

Three panes, chosen because it scales to every exercise type without adding screens.

9.1 Layout

Pane	Contents
Left — Tree	Courses → lessons → exercises. Expand/collapse, click to select. Each exercise shows a type badge so the shape of a lesson is readable at a glance.
Middle — Inspector	Switches on selection. <i>Course</i> : bilingual title, read-only id, lesson/exercise counts, unlock rule. <i>Lesson</i> : bilingual title, explanation. <i>Exercise</i> : type selector, bilingual prompt, the type-specific form, correct/incorrect feedback.
Right — Preview	Renders the exercise exactly as a learner sees it, updating as the admin types, with an Answer key toggle that reveals the solution.

9.2 Per-type editor forms

Type	Form	Preview
Multiple choice	Option rows with EN + SI fields; a radio circle marks the correct option; add / remove	Option cards; correct one ringed green when the key is shown
Fill in the blank	Sentence with , comma-separated accepted answers, optional choice chips	Sentence with an underlined blank plus chips
Drag to order	Tokens listed in correct order with ↑ ↓ reorder and remove	Tokens shown shuffled above a drop zone
Match pairs	Left ↔ right rows; add / remove	Two columns with the right column scrambled
Free text	Accepted answers one per line; ignore-capitalisation and ignore-punctuation toggles	Answer box; accepted list revealed with the key

9.3 Type switching

Changing the type replaces `payload` with `DEFAULT_PAYLOAD[newType]()`. The shared fields — prompt and feedback — survive. Removing the option currently marked correct must clamp `correctIndex` into range rather than leave it dangling.

10. Navigation contract

The dashboard hands off to the editor by deep link:

```
Course Editor?stage=<stageId>
Course Editor?stage=<stageId>&lesson=<lessonId>
Course Editor?stage=<stageId>&lesson=<lessonId>&exercise=<exerciseId>
```


RESOLUTION RULE — A REAL BUG FOUND IN TESTING

A parameter narrows the selection **only if it actually resolves**. An id that was supplied but matches nothing — a stale bookmark, deleted content — is treated as absent, so the editor falls all the way back to a valid selection. The naive implementation conflated “a parameter was supplied” with “it resolved”, and stranded the admin on an empty pane for content that no longer existed. This becomes reachable the moment delete is implemented.

11. Data-visualisation rules

The five exercise-type colours originally chosen for the badges were run through a colour-vision validator and **failed**: green `#3ECF6E` and orange `#FF6B4A` separate by only ΔE 5.2 under deuteranopia (target ≥ 8), and two sat outside the usable lightness band.

The correct fix was not to re-tune five hues. The exercise-mix chart plots **one measure across categories**, which needs **one** hue with direct labels — five hues would encode nothing that the row label does not already say.

Rules to keep:

- Single measure across categories → one hue, direct-labelled, bars scaled to the largest value.
- Type badges keep distinct colours only because a text label always sits beside them; identity never rests on colour alone. Their hues were darkened for contrast against white badge text.
- Status colours (warning / success in Content checks) are reserved, and always ship with an icon and a label.
- Never a dual-axis chart; never a rainbow ramp for magnitude.

12. Keeping the learner roadmaps data-driven

Both learner roadmaps must become **projections** of the same `stages[]` array, not parallel hard-coded lists:

- The Course Map currently hard-codes `X_OFFSETS = [50,66,50,34,50,66,50,34]` — an eight-element array that breaks the moment a ninth course is added. Replace it with a function of `order`, so the zig-zag layout is computed rather than enumerated.
- Planet colours come from `stage.theme`; lock state is `stage.unlock` evaluated against the learner's progress; labels come from `stage.title` in the active language.
- Stage Lessons applies the same pattern one level down — node icon chosen by `lesson.kind`.

Once this holds, adding, renaming, reordering or removing a course is purely a dashboard edit.

13. Build order

Phase 1 — Unify the data, still static. Create one curriculum object. Delete the duplicated `STAGES` arrays. Normalise the three existing question shapes into the single envelope. No backend yet — the goal is to prove the schema against real content.

Phase 2 — Switch routing to slugs. Replace `?stage=2` with `?stage=tenses` everywhere. Cheap now, painful after progress data exists.

Phase 3 — Build the runner and registries. Get MCQ and gap-fill working through the new engine, then add drag-order, match and free-text. If adding one requires touching the runner, stop and fix the abstraction.

Phase 4 — Add the API. Serve the identical shape; the frontend changes from an import to a fetch and nothing else moves.

Phase 5 — Move grading server-side and strip answer keys from the public read path.

Phase 6 — Build the admin UI last. By then the schema is proven, and the editor forms can be generated from the per-type field schemas.

Phases 1–3 are refactors of existing code and already deliver the exercise variety; the backend only becomes necessary at Phase 4.

14. Verification approach

The prototype's page logic is separable from its rendering, which makes it cheaply testable: extract the logic block, run it under Node against a stubbed host and a fake `localStorage`, and assert on the values it produces. Twenty-one such checks currently cover both admin pages. Cases worth keeping in any reimplementation:

- Round trip — an edit saved in the editor is visible on the dashboard, and vice versa.
- Adding two courses in a row must not collide on generated id.
- A type with zero exercises still renders a zero-width bar rather than disappearing.
- An empty curriculum must not divide by zero when computing bar widths.

- Unavailable storage degrades to session-only rather than throwing.
- Stale deep-link ids fall back to a valid selection (§10).
- Removing the option marked correct clamps the index instead of orphaning it.

15. Prototype status

File	Role
<code>curriculum.js</code>	The shared global object, analytics, type registry, load/save helpers
<code>Course Dashboard.dc.html</code>	Statistics, content checks, course/lesson list, add and edit actions
<code>Course Editor.dc.html</code>	Three-pane visual editor, all five type editors, live preview, save

Implemented: the full data model; all five exercise editors; live preview with an answer-key toggle; statistics and content checks; add course and add lesson; deep-linked navigation between the two pages; draft persistence.

Not yet implemented: delete and drag-to-reorder in the tree; the publish flow and its validation gate; server-side grading; the exercise bank (exercises are embedded in lessons rather than referenced); authentication and an admin role.